VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science & Engineering

VITyarthi — Build Your Own Project Evaluation

PROXY FORMATTER & TESTER

A Python-based Proxy Normalization and Connectivity Testing System

Student Name	Atiksh Maurya
Registration / Roll No	26BCE11562
Academic Program	B.Tech Computer Science & Engineering — CSE Core
Course	CSE1021 — Introduction to Problem Solving and Programming
Faculty / Teacher	Mrs. Vaishnavi S.
Project Identifier	ProxyFormatterTester
Submission Platform	VITyarthi Learning Portal
Academic Year	2026–2027

Academic Year 2026–2027 • Department of Computer Science and Engineering

2. Introduction

Proxy lists are commonly received in inconsistent textual formats. Entries may appear as HOST:PORT:USERNAME:PASSWORD, USERNAME:PASSWORD@HOST:PORT, labelled fields, or four-line records. Manual normalization makes validation and later connectivity testing harder and increases the chance of malformed input reaching the testing stage.

Proxy Formatter & Tester is a Python command-line system that automates this workflow. The formatter recognizes supported representations, validates required fields, normalizes valid entries to USERNAME:PASSWORD@HOST:PORT, and removes duplicates. The testing component uses **curl_cffi** to send requests through each formatted proxy, measures elapsed time, records HTTP status where available, classifies common failures, and separates working and failed results.

3. Problem Statement

The project addresses two related problems: inconsistent proxy representations and uncertainty about whether a syntactically valid proxy can actually complete a request. Invalid ports, missing credentials, incomplete labelled records, duplicates, timeouts, connection failures and SSL-related errors can otherwise make a proxy list difficult to use.

The proposed solution is a sequential pipeline: **parse** → **validate** → **normalize** → **test** → **classify** → **report**. The project processes only proxy data supplied by the user and does not provide proxy acquisition or credential generation.

4. Functional Requirements

Module	Functionality
1. Input & Parsing	Read proxy records and recognize supported direct, URL-style, labelled and four-line formats.
2. Validation & Normalization	Validate required values and port range; output USERNAME:PASSWORD@HOST:PORT.
3. Duplicate & File Processing	Remove duplicate normalized entries and write proxies.txt.
4. Connectivity Testing	Accept a test URL and perform HTTP/HTTPS requests through each proxy.
5. Reporting	Measure time, record status/errors, and write working.txt, fail.txt and proxy_results.txt.

5. Non-Functional Requirements

Requirement	Implementation Goal
Performance	Avoid repeated parsing and expose request elapsed time for each proxy.
Reliability	Continue processing remaining entries after malformed input or request exceptions.
Usability	Provide clear prompts, progress messages, readable errors and a final summary.
Maintainability	Separate parsing, validation, testing, error classification and orchestration into functions.
Error Handling	Handle missing files, invalid records, timeouts, connection and SSL-related failures.
Resource Efficiency	Use standard file I/O and write test results as the batch progresses.

6. System Architecture

The current implementation follows a simple layered pipeline. Raw proxy records enter the formatter, valid normalized records are written to an intermediate file, and the tester consumes that file to produce classified outputs.

Figure: Logical Processing Layers

Recommended repository structure

```
proxy-formatter-tester/
■■■ formatter.py
■■■ proxy_tester.py
■■■ requirements.txt
■■■ README.md
■■■ statement.md
■■■ tests/
■ ■■■ test_formatter.py
■ ■■■ test_proxy_tester.py
■■■ docs/diagrams/
■■■ PROJECT_REPORT.pdf
```

7. Design Diagrams

7.1 Use Case Diagram

Actor	Use Case	Result
User	Prepare raw proxy input	Input file available
User	Run formatter	Normalized proxies generated
User	Provide test URL	Testing target selected
User	Run proxy tester	Each proxy is tested
User	Inspect result files	Working/failed/detailed results

Figure 7.1: Use-case view of the user interacting with the formatter and tester.

7.2 Workflow Diagram

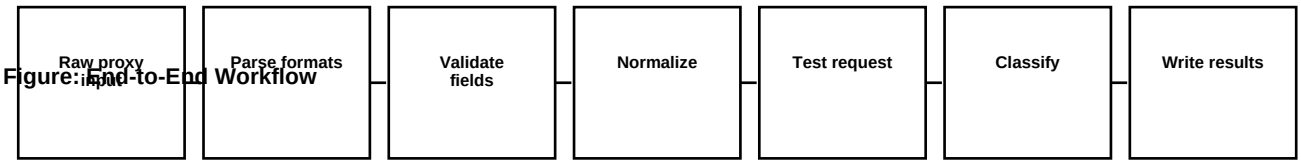

Figure: End-to-End Workflow

Figure 7.2: Operational workflow from raw input through testing and result generation.

7.3 Sequence Diagram

Step	Component	Operation
1	User	Prepare input and start formatter
2	Formatter	Read and strip non-empty lines
3	Formatter	Parse and validate proxy representation
4	Formatter	Normalize and write proxies.txt
5	Tester	Read proxies and test URL
6	curl_cffi	Send request through proxy
7	Tester	Classify result and write outputs

Figure 7.3: Major method and data hand-offs.

7.4 Class / Component Diagram

Component	Responsibilities
formatter.py	format_proxy(), clean_label(), main(); parsing, validation, normalization and deduplication.
proxy_tester.py	format_proxy(), classify_error(), test_proxy(), main(); URL handling, request testing and reporting.
curl_cffi	HTTP client used for proxy-routed requests and browser-style request impersonation.
Output files	proxies.txt, working.txt, fail.txt and proxy_results.txt form the file-based data flow.

Figure 7.4: Component-level view of the current implementation.

7.5 Storage / File Design

File	Role	Contents
unformat_proxy.txt	Raw input	User-supplied proxy records
proxies.txt	Normalized data	USERNAME:PASSWORD@HOST:PORT
working.txt	Successful results	Proxies with successful response
fail.txt	Failed results	Proxies with request failure
proxy_results.txt	Detailed report	Timestamp, URL, status, time and errors

Figure 7.5: File-based storage design.

8. Design Decisions & Rationale

• **Layered processing:** Formatting is kept separate from network testing so parsing errors and connectivity failures can be distinguished.

• **Standardized output:** USERNAME:PASSWORD@HOST:PORT provides a consistent hand-off between the two stages.

• **curl_cffi:** The current tester uses curl_cffi for proxy-routed HTTP/HTTPS requests and Chrome impersonation.

• **Separate result files:** Working, failed and detailed outputs support immediate reuse and later inspection.

• **Defensive error handling:** Common request failures are converted into readable labels.

9. Implementation Details

9.1 Proxy Formatting

The formatter first tries to parse a complete proxy on one line. It supports USERNAME:PASSWORD@HOST:PORT, URL-prefixed forms, HOST:PORT:USERNAME:PASSWORD, labelled fields such as IP/HOST/PORT/USERNAME/PASSWORD, and four consecutive unlabeled lines. Valid ports are numeric and restricted to 1–65535. Duplicate normalized entries are removed while preserving order.

9.2 Proxy Testing

The tester reads proxies.txt and accepts a test URL from the user. If the URL has no HTTP/HTTPS scheme, it adds https://. The proxy is supplied for both HTTP and HTTPS requests. The current source sets a 15-second timeout and Chrome impersonation.

9.3 Error Classification

Successful requests record proxy, HTTP status and elapsed time. Request exceptions are classified where possible as timeout, proxy connection failure, SSL verification failure, or connection failure; unexpected exceptions are also recorded.

10. Screenshots & Demonstration Results

For final submission, include actual screenshots from the completed implementation: formatter execution, generated proxies.txt, tester progress, working.txt/fail.txt, and the final summary. This report intentionally does not invent measured network results.

Demonstration Output

Output	Expected Evidence
Formatter	Formatted count, skipped count and output path
Tester	Per-proxy WORKING/FAILED status and timing/error
working.txt	Successful proxy entries
fail.txt	Failed proxy entries
proxy_results.txt	Detailed timestamped test results

11. Testing Approach

Testing should cover deterministic formatter behavior separately from network-dependent connectivity testing. Formatter tests can use fixed strings; network tests should use controlled test URLs and should be interpreted as environment-dependent rather than permanent guarantees.

Category	Scenario	Expected Result
Parser	HOST:PORT:USER:PASS	Correct normalized proxy
Parser	USER:PASS@HOST:PORT	Correct normalized proxy
Parser	URL-prefixed input	Correct extraction
Labels	IP/HOST/PORT/USERNAME/PASSWORD	Correct grouped record
Validation	Invalid port or missing field	Entry rejected
Deduplication	Repeated normalized entry	One output entry
File handling	Missing input file	Readable error and graceful exit
Network	Timeout/connection failure	Failure classified and recorded
Output	Mixed working/failed batch	Correct files generated

12. Challenges Faced & Solutions

• **Multiple input formats:** A layered parser handles direct, URL-style, labelled and four-line records.

• **Malformed entries:** Unrecognized records are skipped and counted rather than terminating the complete formatting run.

• **Network variability:** Syntax validity is separated from actual request success.

• **Error readability:** Exception messages are mapped to human-readable categories where possible.

• **Credential safety:** Real proxy lists and generated result files should be excluded from public Git repositories.

13. Learnings & Key Takeaways

• **Defensive programming:** Validation reduces downstream failures.

• **Modular programming:** Separate functions make the workflow easier to maintain and test.

• **File processing:** Inconsistent text can be transformed into a predictable machine-readable representation.

• **Exception handling:** Network applications must tolerate operational failures.

• **Documentation:** Diagrams, tests, README and a report turn a script into a documented academic project.

14. Future Enhancements

• **Command-line arguments:** Accept input/output files, test URL and timeout directly from the terminal.

• **Controlled concurrency:** Test larger proxy lists more efficiently while retaining safe resource limits.

• **CSV/JSON export:** Provide structured output for later analysis.

• **Configuration file:** Move timeout, impersonation and paths out of source code.

• **Automated test suite:** Add comprehensive unittest tests for parsing, validation and result generation.

• **Statistics:** Report success rate, average latency and failure-category distribution.

• **Protocol-aware testing:** Clearly separate and document supported proxy protocols.

• **GUI:** Provide an optional graphical interface for users who do not prefer the command line.

15. References

1. Python Software Foundation. Python 3 Documentation — pathlib and urllib.parse.

2. curl_cffi Documentation — HTTP requests and browser impersonation.

3. Vellore Institute of Technology. VITyarthi — Build Your Own Project Evaluation: General Project Instructions & Submission Guidelines.

4. Project source files: formatter.py and proxy_tester.py.

Submission Note

This report follows the section sequence and presentation style of the supplied reference project report. Before final submission, add actual screenshots and measured test results, complete the required 5–10 meaningful source modules/files and automated tests, and ensure README.md and statement.md are included in the GitHub repository.